Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: 05.11.2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/Sarah270809/CSA0415-operating-system

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

OPERATING SYSTEMS – ASSIGNMENT 1

CO1: Evaluate and Justify the Operating-System Design of a Smart Hospital

Student Name: S. Sarah Khan Register Number: 1965260116

Submission Date: 05.11.2026

GitHub Repository Link: https://github.com/Sarah270809/CSA0415-operating-system

Note: Replace measurement placeholders with outputs obtained by executing the programs on your own Linux system. Do not submit estimated timings as measured results.

TASK 1 – Evaluate the Requirements of Each Computing Environment

The hospital should not use one identical operating-system configuration for every device. The consequences of delay, failure, power use and data exposure differ greatly between clinical monitors, mobile tablets, workstations and servers. The requirements below are ranked from highest to lowest priority for each environment.

Environment	Ranked requirements and justification
Bedside patient monitors	1. Deterministic response: an abnormal reading must trigger an alarm within 10 ms. 2. Reliability and safe recovery: missed alarms can harm patients. 3. Correct sensor I/O and fault detection. 4. Security and integrity of readings. 5. Low memory and power use. A small real-time OS is preferred; nonessential services should be excluded.
Nurses' handheld tablets	1. Patient-data confidentiality and authentication. 2. Battery life for a full shift. 3. Reliable wireless communication and offline handling. 4. Usable interface and responsive applications. 5. Manageable updates. A mobile/handheld OS with encryption, screen locking and controlled background activity fits best.
Doctors' desktop workstations	1. Secure access to records and clinical applications. 2. Interactive response and usability. 3. Compatibility with imaging and hospital software. 4. Reliability and update management. 5. Resource efficiency. A desktop OS provides a mature interface, drivers and application support.
Central patient-record database	1. Availability and data integrity. 2. Access control, auditing and backup. 3. High throughput and predictable storage I/O. 4. Recovery and maintenance without extended downtime. 5. Scalability. A server OS with clustering, replication and monitored fail-over is suitable.
MRI imaging server	1. High throughput and parallel processing. 2. Sufficient RAM and fast storage. 3. Reliable transfer of large image files. 4. Security and traceability. 5. Efficient scheduling. A multiprocessor server OS and DMA-capable I/O are appropriate.
Partner-hospital connection	1. Confidentiality and authenticated communication. 2. Availability and data consistency. 3. Network fault recovery. 4. Interoperability and audit logs. 5. Scalable service capacity. A distributed architecture with encrypted, authenticated APIs is preferred.

Batch systems taught the value of efficient queued processing, but cannot guarantee immediate alarms. Time-sharing supports many concurrent users on desktops and servers. Personal-computer operating systems offer interactive interfaces for clinicians. Mobile operating systems manage battery, wireless links and background work. For a single general-purpose OS everywhere, administration might be more uniform, but the monitor's worst-case scheduling delay would not be guaranteed: ordinary time-sharing can delay a task behind other work, whereas a properly configured real-time OS can provide a bounded deadline under stated assumptions. General-purpose installations also consume more memory/storage on small devices and increase configuration burden. Therefore, standardise management where practical, not the OS design at the expense of patient safety.

TASK 2 – Judge the Type of Operating System for Each Environment

A suitable design assigns each environment the OS type that matches its workload rather than forcing all environments into one category.

Environment	Recommended type	Rejected alternative and reason
Monitors	Embedded hard/firm real-time OS, with a verified 10 ms alarm deadline	A normal desktop OS does not inherently guarantee bounded worst-case latency.
Tablets	Handheld/mobile OS	A desktop OS typically consumes more power and has a less suitable touch/mobile management model.
Workstations	Desktop, time-sharing OS	A real-time OS is unnecessary for normal interactive office workloads.
Database	Server OS deployed as a cluster with fail-over	One large standalone machine remains a single point of failure.
MRI server	Multiprocessor server OS; symmetric multiprocessing (SMP) where processors share memory and OS scheduling	Asymmetric multiprocessing assigns fixed roles and may underuse processors or complicate load balancing unless a specialised workload benefits from it.
Partner hospitals	Distributed system with secure network services	A single-host model cannot naturally provide shared records across independent sites.

Mainframes are designed for large-scale central transaction workloads; the hospital database has similar reliability and throughput needs, though a modern clustered server is the practical choice. Desktop and handheld systems prioritise interaction and energy efficiency respectively. Multiprocessor systems improve parallel processing. Distributed systems coordinate services across networked machines. A cluster combines machines to improve availability; database fail-over should detect a node failure and switch service within seconds, with tested replication and recovery. Compared with one large multiprocessor server, a cluster costs more to configure and operate but avoids one machine being a single point of failure. Throughput can increase with processor count, but not linearly forever because of serial work, locking and memory bandwidth (Amdahl's law). Space complexity is driven by memory per node and replicated data; a cluster may require extra memory/storage for redundancy.

TASK 3 – Evaluate Operating-System Structures

Operating-system structure affects failure containment, speed, and the ability to update components safely.

Structure	Evaluation for the hospital
Monolithic/simple	Most OS services run in kernel space. System calls can be fast, but a faulty driver or service may crash the whole kernel. Less suitable for safety-critical monitor fault containment.
Layered	Clear layers simplify reasoning and maintenance, but strict layer boundaries can add overhead and limit flexibility.
Microkernel	Keeps only essential functions in the kernel and moves drivers/services to separate user-space servers. Better isolation and fault containment, although message passing and context switches can add latency.
Modular kernel	A core kernel supports loadable modules, such as drivers. It balances performance and extensibility, but a faulty kernel module can still compromise the kernel.
Hybrid	Combines ideas from monolithic and microkernel designs to balance speed and modularity.

Recommended choice: use a small, validated microkernel-style or purpose-built real-time OS for bedside monitors because isolation and predictable response are important. For the database server, a modular Linux server kernel is practical because it provides mature storage, networking, monitoring and clustering support with good performance. Linux is generally monolithic with loadable modules; Windows is commonly described as hybrid; Android uses the Linux kernel plus Android framework and services; FreeRTOS is a small embedded real-time kernel, while QNX is microkernel-based. These labels are simplified descriptions and actual implementations vary by version and configuration.

If a monitor instead used a monolithic kernel, service requests might require fewer message-passing steps and therefore have lower average overhead. However, the kernel may occupy more memory and, more importantly, a faulty driver could crash or corrupt the whole system. A microkernel may require extra inter-process messages, but a carefully designed real-time system can still meet 10 ms; the deadline must be verified through measurement and worst-case analysis, not assumed from the structure alone.

TASK 4 – Evaluate the Services an Operating System Must Provide

OS services should be selected according to each device's job and risk. The monitor needs only a small, dependable set of services, while the database requires a broad set of protection and recovery features.

Service	Monitors	Tablets/workstations	Database/imaging servers
User interface	Minimal alarm/status display	Full touch or desktop interface	Administration and monitoring interface
Program execution	Essential monitoring and alarm tasks	Clinical apps and approved processes	Database, imaging and maintenance services
I/O and error detection	Sensor, display, alarm output; watchdogs	Touch, storage, wireless; device error handling	Storage/network controllers, checksums and error reporting
File system	Small configuration/log storage if required	Encrypted local storage and managed files	Reliable file system, logs, backup and recovery
Communication	Only required clinical links	Secure Wi-Fi and hospital services	Secure network/API, replication and partner links
Resource allocation/accounting	Predictable CPU and memory; minimal accounting	Battery-aware scheduling and device management	CPU, RAM, storage and network allocation; audit and usage monitoring
Protection/security	Secure boot, least privilege, authenticated updates	Encryption, authentication, remote lock/wipe	Role-based access, least privilege, audit, encryption, backups

Services that can be removed from a bedside monitor include a general-purpose desktop shell, print services, unrelated networking protocols, app stores, and nonessential background services. Keep the scheduler, timer, sensor and alarm drivers, memory protection, watchdog/error handling, minimal secure communication and essential logging. The database must retain process management, reliable file/storage I/O, network communication, access control, auditing, error detection, resource allocation, backup and recovery. Patient records should be protected using authenticated identities, role-based access, process isolation, file/database permissions, encryption and audit logs.

A full desktop OS on tablets would add background services, larger storage and memory requirements, longer startup, more patching and more battery drain. A reduced mobile OS can restrict background work and use power-management states. This is a space/time/power trade-off: additional services increase footprint and work, while excluding unnecessary services improves endurance and reduces attack surface.

TASK 5 – Examine System Calls

A system call is the controlled interface through which a user program requests a privileged operating-system service. Categories include process control (fork, exec, wait), file management (open, read, write, close), device management (device-specific operations), information maintenance (getpid, clock queries), communication (pipes, sockets) and protection (permissions and identity operations).

For read(fd, buffer, count), the program usually calls a C library wrapper. The wrapper places the system-call number and arguments in the architecture-defined registers or memory locations and executes a trap/syscall instruction. The CPU switches from user mode to kernel mode, the kernel validates the descriptor, pointer and requested operation, then obtains data from a file/device or cache. The kernel returns a byte count or error,

switches back to user mode, and the library reports the result. Parameters may be passed in registers, a memory block referenced by a register, or the stack, depending on the ABI.

Example evidence command: `strace -e trace=openat,read,write,close ./program`. It prints system calls and return values, helping verify which OS operations occurred. `strace -c ./program` summarises counts and time. For buffered stdio, fopen/fread/fwrite perform library-level buffering; many small application reads/writes may be combined into fewer kernel calls. For n bytes and a library buffer of roughly 4096 bytes, calls can be approximately O(n/4096), rather than O(n/b) when direct calls use a small b-byte buffer. The library buffer adds O(4096) space per active stream in this example. The best choice depends on measured workload, not an assumed universal optimum.

TASK 6 – Evaluate the Cost of System Calls

System calls cost more than ordinary function calls because they cross the user/kernel boundary, validate arguments and permissions, and may trigger scheduling, device work or cache disruption. The cost is workload-dependent; a call can also block while waiting for I/O.

Completed code for the supplied TODO is shown below. It handles partial writes and counts each read call. The fixed buffer is 8192 bytes, so `bufsize` must not exceed that capacity.

```
    while (1) {
        size_t want = bufsize;
        if (want > sizeof buf) want = sizeof buf;
        n = read(in, buf, want);
        calls++;
        if (n == 0) break;
        if (n < 0) { calls = -1; break; }
```

```
   ssize_t written = 0;
   while (written < n) {
      ssize_t w = write(out, buf + written, (size_t)(n - written));
      if (w < 0) { calls = -1; break; }
      written += w;
   }
   if (calls < 0) break;
}
close(in);
close(out);
return calls;
```

For the provided 10,000-byte file, the expected read counts are 10,001 with a 1-byte buffer, 21 with 512 bytes, and 4 with 4096 bytes. The extra call in each case detects end-of-file. This shows that larger buffers sharply reduce call overhead. The 512-byte case uses about ceil(10000/512)+1 = 21 calls; the 4096-byte case uses ceil(10000/4096)+1 = 4 calls. Time complexity for copying is O(n), while auxiliary space is O(b) for buffer size b. A memory-mapped approach uses mmap/munmap calls rather than a read/write call per buffer, and can reduce explicit copying between kernel and user space. However, page faults still occur, mappings consume virtual address space, and mapping huge files may create memory pressure. For very large backups, benchmark buffered I/O against mmap on the actual filesystem and storage.

TASK 7 – Evaluate Process Control System Calls

A shell starts a command by creating a child process with fork(), replacing the child's program image with exec(), and collecting the child's completion with wait()/waitpid(). exit() terminates a process and supplies an exit status. fork() returns 0 in the child and the child PID in the parent; exec() replaces the current process image if successful.

Completed TODO for job 1:

```
pid_t pid;
int status;
pid = fork();
if (pid < 0) {
    perror("fork");
    return 1;
}
if (pid == 0) {
    execlp("expr", "expr", "6", "*", "7", (char *)NULL);
    perror("execlp");
    _exit(127);
}
if (waitpid(pid, &status, 0) < 0) {
    perror("waitpid");
    return 1;
}
if (WIFEXITED(status))
    printf("job 1 exit status %d\n", WEXITSTATUS(status));
else
    printf("job 1 ended abnormally\n");
```

The expected normal output is `shell: running job 1`, then `42`, then `job 1 exit status 0`; job 2 prints status 3. `fflush(stdout)` before fork prevents buffered text from being duplicated by both parent and child when stdout is redirected to a file. With no flush, the output buffer is copied during fork. If the child calls `exit()`, it flushes its inherited copy; the parent later flushes its own copy, so "shell: running job 2" may appear twice. `_exit()` avoids flushing inherited stdio buffers.

A zombie is a terminated child whose exit status has not yet been collected; waitpid() reaps it. An orphan is a child whose parent exits first; it is adopted by a system process that can reap it. For 1,000 commands, fork/exec gives each command process isolation but creates process-management and address-space overhead; copy-on-write avoids eagerly copying all memory pages. posix_spawn() may reduce overhead for simple launches. Threads start with less separate memory because they share an address space, but commands run as threads have weaker isolation: a crash or memory corruption can affect the whole process. Process creation time should be measured on the target Linux system rather than invented.

TASK 8 – Evaluate CPU States and Protection

The CPU uses user mode for applications and kernel mode for trusted OS code. A mode bit indicates the current privilege level. Privileged instructions—such as disabling interrupts or changing page tables—are restricted to kernel mode. A system call or exception transfers control to the kernel through a controlled entry point; hardware interrupts notify the CPU about external events, and timer interrupts let the OS regain control from a running task.

For the patient monitors, user-mode applications must not be allowed to disable interrupts, modify hardware directly or read arbitrary memory. If any program could disable interrupts, an alarm task might never run and the monitor could miss its 10 ms deadline. Memory protection through virtual memory and page permissions prevents one process from reading or overwriting another process's memory. The kernel also validates pointers and permissions at system-call boundaries.

If the sensor were polled every P milliseconds, a newly abnormal reading could wait nearly P milliseconds before being noticed, plus processing delay. With interrupt-driven input, the delay is closer to interrupt latency and handler/scheduling delay, provided interrupts are not masked too long and higher-priority work is bounded. Polling is simple and may be appropriate at very short intervals, but repeated checks waste CPU cycles and battery power when no new reading exists. Interrupts are generally preferable for efficient event detection; the real-time deadline must still be verified under worst-case load.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems

The memory hierarchy trades speed, capacity and cost. Registers are the smallest and fastest; CPU caches are very fast but small; RAM is larger but slower; SSDs and hard disks provide persistent storage with much higher latency; network storage adds communication and remote-service latency. Typical values vary by hardware, so the following are approximate orders of magnitude rather than guarantees.

Level	Typical access time (approx.)	Hospital use
CPU registers	Sub-nanosecond to a few ns	Current instructions and immediate operands
L1/L2/L3 cache	About 1–tens of ns	Frequently used code/data
Main memory (RAM)	About 50–150 ns	Active database cache, application data, image-processing buffers
NVMe/SATA SSD	Tens of µs to sub-ms	Patient records, recent MRI images, logs and fast local storage
Hard disk	Several ms	Large archive and less frequently accessed images
Network/cloud storage	Network- and service-dependent; often ms or more	Replicated records and long-term shared archive

Patient records should reside on reliable database storage with backups and replication; frequently accessed indexes and records can be cached in RAM. MRI images need high-throughput local SSD/cache for processing and durable storage/archives afterwards. Monitor readings and alarms need prompt local processing, with selected readings securely transmitted and logged.

Programmed I/O makes the CPU repeatedly check and transfer data; it is simple but wastes CPU. Interrupt-driven I/O lets the device notify the CPU when service is needed. DMA transfers blocks between a device and memory with less CPU involvement, making it suitable for large MRI image transfers. If a 500 MB image generated one interrupt per word, the interrupt count would be enormous: for 4-byte words, 500,000,000 / 4 = 125,000,000 interrupts (about 131,072,000 if 500 MiB). Handling that many interrupts would consume substantial CPU time. DMA instead uses a small number of transfer completions/interrupts and block buffers. DMA needs appropriately sized memory buffers and safe ownership/coherency management; interrupt-driven per-word I/O can use smaller transfers but has much greater CPU overhead. Caching speeds record lookups when data is reused, but stale or improperly invalidated cache entries must not compromise record correctness.

TASK 10 – Measure and Compare

Measurements should be collected on the Linux system used for the assignment. Because execution time depends on CPU, kernel, storage, compiler, background activity and virtualisation, actual measured values must be inserted rather than fabricated.

Experiment	Command / method	Expected interpretation
Copy buffer sizes	`strace -c ./copy_program` and print the program's own read counter	Read calls should drop sharply as buffer size rises; compare application counter with strace totals, noting that strace also sees calls made by program setup and libraries.

User/kernel time	`time ./copy_program` (or `/usr/bin/time -v ./copy_program`)	User time measures application CPU work; system time measures kernel CPU work. Wall time also includes waiting.
1,000 processes	Write a loop using fork() and waitpid(), then time it	Higher per-task isolation but more process and address-space bookkeeping.
1,000 threads	Write a pthread_create()/pthread_join() loop and time it	Often faster/lighter than processes because threads share the process address space, though each thread still needs a stack and kernel resources.

Example measurement table to fill in after execution:

```
Buffer (bytes) | Program read() calls | Real time | User time | System time
1              | 10001                | measured  | measured  | measured
512            | 21                   | measured  | measured  | measured
4096           | 4                    | measured  | measured  | measured
```

The call counts for the supplied 10,000-byte input are derived from the program's expected output; timings must be measured locally. Use `gcc -std=c11 -Wall -Wextra -O2 copy.c -o copy` and `strace -c ./copy`. Compare 1,000-process and 1,000-thread creation using the same machine, similar work, repeated trials and the median. Threads commonly need less memory per worker because code, heap and open files are shared, while processes have separate address spaces (with copy-on-write after fork). Exact creation times and memory depend on stack size, libraries and kernel limits. Discuss whether measurements support the hypothesis, and report any environmental limitations.

TASK 11 – Recommend and Defend the Operating-System Plan

The recommended design is a purpose-built real-time layer for bedside monitors, a mobile OS for tablets, desktop OSs for clinical workstations, and a secure clustered server platform for the patient database. The MRI imaging service should run on a multiprocessor server with fast storage and DMA-capable I/O. Partner-hospital access should use secure distributed services and audited interfaces.

Environment	Type/structure and essential safeguards
Monitors	Embedded real-time OS; minimal services, bounded high-priority alarm task, watchdog, protected memory and validated sensor drivers.
Tablets	Managed mobile OS; encryption, authentication, device management, restricted background activity and secure wireless communication.
Workstations	Supported desktop OS; role-based access, endpoint protection, automatic updates and screen locking.
Database	Server OS with modular kernel in a fail-over cluster; replication, backups, audit logs, access controls, monitoring and tested recovery.
MRI server	Multiprocessor server OS with SMP, sufficient RAM, fast SSD storage and DMA for large transfers.
Three-hospital group	Distributed services with encrypted links, identity federation, audit trails, data-sharing agreements and clear outage/offline procedures.

The main risks are configuration complexity, patching and compatibility across several platforms, network outages, cluster split-brain or replication errors, and cyberattacks. Mitigate these with a common security baseline, central monitoring, tested backups, least privilege, signed updates and routine disaster-recovery exercises. The plan remains suitable for three hospitals if shared services use standard secure interfaces and fail safely when remote systems are unavailable.

Moving every environment into virtual machines on two large servers could simplify deployment and improve consolidation, but it is not appropriate for the bedside alarm control loop unless the entire virtualisation stack is validated for bounded latency and failure isolation. Hypervisor scheduling, shared CPU/memory contention and host failure can add delay or outage. Virtualisation also consumes RAM and processor resources and two hosts

may still be a shared failure domain. It can be useful for database, web/API, test and administrative workloads with high availability, but local real-time monitor control should remain independent. Final performance claims must be supported by the system's own measurements.

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)

Decision: MODIFY the original design, retaining its separation of real-time, mobile, desktop and server roles while extending it for remote devices and cloud-hosted records.

Part	Evaluation and revised design
A. Impact analysis	Home monitoring introduces untrusted networks, variable latency, intermittent connectivity and device diversity. Cloud records add identity federation, network security, data residency/availability and shared-access concerns. The embedded real-time OS, communication services, authentication, encryption, logging and server architecture are affected.
B. Evaluate original design	The local real-time monitor design remains suitable for immediate alarms because it does not depend on internet response. The original central database design needs modification: remote monitoring and partner access require distributed services, replication, stronger identity controls and a cloud availability plan. Internet latency cannot be guaranteed below 10 ms, so emergency alarm decisions must not rely solely on a cloud round trip.
C. Adaptation	Use an embedded real-time OS on home devices with local alarm logic, secure boot, signed updates, encryption and store-and-forward buffering during outages. Transmit readings through authenticated encrypted channels to a gateway/API. Host records in a secure cloud/distributed database with redundancy, backups, access logging, role-based access and encrypted links to partner hospitals. Apply least privilege, key rotation, monitoring and recovery testing.
D. Evidence-based defence	Retaining local real-time control protects the 10 ms alarm requirement even when the internet is slow or unavailable. Cloud services improve scale and sharing, but increase dependency on networks and providers and expand the attack surface. Redundancy and backups improve availability but cost more and require consistent data handling. Pilot the system, measure latency and outage recovery, test security and fail-over, and review results before full deployment.

Conclusion: A complete redesign is unnecessary; a controlled modification is safer and more economical. Keep time-critical monitoring local, and use the cloud for secure aggregation, long-term storage and cross-hospital sharing. A remote reading should include a timestamp, device identity and integrity protection. During an outage, the home device should buffer data and follow its validated local alarm policy; it should clearly indicate communication failure rather than silently losing readings. The design should be verified with measured network delay, recovery tests, security testing and clinical risk review.

ORIGINAL RUBRIC / MARK DISTRIBUTION

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design **8 Marks**

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3

Task	Bloom's Level	Marks
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.